5-4 · Spark DataFrame과 SparkSQL 데이터 가공 · 실습 1H

SparkSQL로 집계와 뷰 만들기

같은 가공을 SQL 로 — 계획 대조와 저장

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 임시 뷰 둘을 등록하고 SHOW VIEWS IN default 로 확인한다

② 같은 집계를 SQL 로 써서 API 의 sales 와 같은지 대조한다

③ 두 explain() 이 SortMergeJoin 기반 같은 모양임을 확인한다

④ CTE 로 단계를 나누고 결과를 Iceberg 테이블로 저장한다

앞 메뉴의 셀과 orders · products · sales 변수를 그대로 이어서 씁니다

실습 흐름

여섯 단계 — 뷰 등록에서 미니 과제까지

셀을 다시 띄웠다면 NameError — 앞 메뉴 1~3단계와 6단계(sales)부터 다시

1. 임시 뷰	2. SQL 집계	3. 계획 대조	4·5. CTE·저장	6. 미니 과제
orders_v products_v	API 결과와 대조	explain() 둘 나란히	누적 매출 Iceberg	left_anti 비중 %

1·2단계

뷰를 등록하고 같은 집계를 SQL 로

뷰 등록	orders.createOrReplaceTempView("orders_v")
확인	spark.sql("SHOW VIEWS IN default").show()
SELECT	p.name, SUM(o.qty * p.price) AS total_amount, SUM(o.qty) AS total_qty
FROM	orders_v o JOIN products_v p ON o.product_id = p.product_id
GROUP BY	p.name ORDER BY total_amount DESC

결과 monitor 930000·3 / keyboard 315000·7 / mouse 69000·3 — 앞 메뉴의 sales 와 같아야

일부러 실패시키기

IN default 를 빼면 — AssertionError

SHOW VIEWS IN default 결과: namespace 빈칸 · orders_v · products_v · isTemporary true

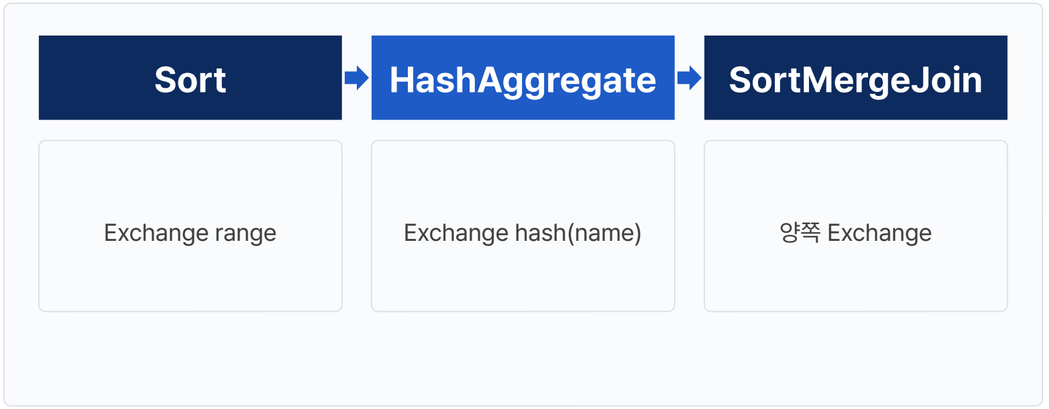
1. IN default 없이	2. 오류	3. 원인	4. 뷰는 무사	5. 처방
spark.sql("SHOW VIEWS")	AssertionError: assertion failed	Iceberg 확장이 뷰 명령을 가로챔	임시 뷰는 정상 등록됨	SHOW VIEWS IN default

3단계

두 explain() 을 나란히 — 같은 모양

sql_sales.explain()

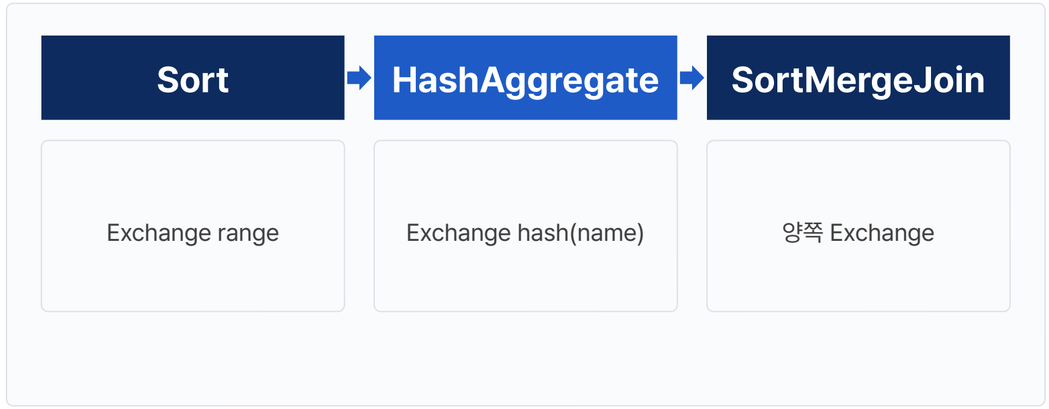
집계 표기 sum((qty * price))



VS

sales.explain()

집계 표기 sum(amount) — 다른 곳은 이 정도



4단계

CTE 와 지연 평가 — q 와 q.show()

1. **q = spark.sql("""WITH enriched AS (...), daily AS (...) SELECT ...""")**
   아무 출력 없이 >>> — 계획만 세웠고 실행은 아직(지연 평가)

2. **q**
   DataFrame[order_date: date, revenue: bigint, cum_revenue: bigint] 한 줄

3. **q.show()**
   08-24 113000·113000 / 08-25 1155000·1268000 / 08-26 46000·1314000

4. **WARN WindowExec: No Partition Defined ...**
   OVER 에 PARTITION BY 가 없다는 경고 — 누적 합계라 맞는 동작

5단계

Iceberg 로 저장 — 세 가지 + SQL CTAS

방식	한 줄	동작
createOrReplace()	sales.writeTo("lake.db.product_sales") .createOrReplace()	없으면 만들고 있으면 통째로 교체
append()	.writeTo(...).append()	뒤에 붙임
overwritePartitions()	.writeTo(...).overwritePartitions()	해당 파티션만 교체
SQL 로도 — CTAS	CREATE OR REPLACE TABLE lake.db.daily_revenue USING iceberg AS SELECT ...	113000 · 1155000 · 46000 4-4 Trino CTAS 형태

처음 만들 때 WARN version hint 스택이 지나감(무시) — MinIO warehouse/lake/db/product_sales/ 에 parquet 1개

6단계

미니 과제 셋 — 5-5 데이터 마트의 뼈대

1. **날짜별 매출을 API 와 SQL 로 각각**
   결과를 대조 — 5-5 에서 만들 데이터 마트의 뼈대

2. **주문이 한 건도 없는 상품**
   products.join(orders, on="product_id", how="left_anti") — 0행

3. **상품별 매출 비중(%)**
   monitor 70.8 · keyboard 24.0 · mouse 5.3 — 합 100.1(반올림)

left_anti

제목 줄만

0행이면 정답

확인 포인트

이 1시간이 끝날 때 확인할 것

1. SHOW VIEWS IN default 에 orders_v · products_v
2. SQL 집계 결과가 API 의 sales 와 완전히 같다
3. 두 explain() 이 SortMergeJoin 기반 같은 모양이다
4. q.show() 에 날짜별 매출과 누적 매출이 나온다
5. product_sales · daily_revenue 조회 · MinIO 에 parquet
6. left_anti 0행 · 비중 70.8 · 24.0 · 5.3

5-4 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

Pandas · PySpark · SQL 세 도구를 한 표로 놓고, 5-5 파티셔닝 준비를 확인합니다

- ✓ 세 도구 대조표 한 장
- ✓ SHOW TABLES IN lake.db 로 테이블 셋
- ✓ sales 파티션 1 · 셔플 파티션 200
- ✓ exit() 뒤 docker compose stop

가져갈 것

셸 그대로 · sales 변수 살아 있게